



27. Independent Component Analysis

Separating Mixed Signals, Revealing Independent Sources

Previous Section:

 [26. Principal Component Analysis](#)

Contents:

[1. Independent Component Analysis – Statistical Independence](#)

[2. Independent Component Analysis – The Cocktail Party Problem](#)

[3. Independent Component Analysis – A Simple Dataset \(Voice Signals\)](#)

[4. Independent Component Analysis with Python](#)

[Summary](#)

[References](#)

This section is about **Independent Component Analysis (ICA)**. And at first glance, it might feel a bit strange. Because unlike most Machine Learning models we've seen so far. ICA is not trying to predict something. It's not classifying. It's not regressing. It's not even clustering in the usual sense. So the question becomes: **What is ICA actually trying to do?**

Let's slow down for a moment. Imagine you're in a crowded room. Multiple people are talking at the same time. Voices overlap. Sounds mix together. And what reaches your ears is not a single clean signal, but a combination of many.

Now here's the interesting part: Even in that chaos, your brain can focus. You can isolate one voice. Follow one conversation. Almost as if you are separating the signals.

ICA tries to do exactly that. Not with voices, but with data. Instead of assuming that what we observe is clean and independent. ICA starts with the opposite idea:

- What we see is often a mixture.
- What we measure is often combined.
- What we receive... is not the original source.

So instead of learning patterns directly from data, ICA asks a much deeper question: ***“Can we recover the original signals... from what we observe?”***

And that’s what makes it special. Most models go from input to prediction. ICA goes from mixture back to source.

But here’s the catch. This only works under a very specific assumption: The original sources must be **independent**. Not just uncorrelated. Not just different. But statistically independent. And that idea is the foundation of everything that follows.

1. Independent Component Analysis – Statistical Independence

Now let’s finally make it precise. What exactly is **Independent Component Analysis (ICA)** doing?

At its core, ICA is trying to solve one very specific problem:

- **Take mixed signals → and recover the original, independent sources.**
- But not just any sources. They must be **independent** → And importantly, **non-Gaussian**

Let’s connect this to something you already know. You’ve seen **PCA** before. PCA looks at data and says: “Let me find directions with the highest variance” → “Let me make components uncorrelated”. That’s useful. Very useful. But ICA looks at PCA and says: “That’s not enough.”

Because **uncorrelated does NOT mean independent**. So ICA pushes further. It doesn’t just want components that don’t correlate. It wants components that behave like they have **nothing to do with each other at all**.

What does “independent” actually mean? Let’s slow down and define it properly. Two signals A and B are **statistically independent** if:

$$P(A \cap B) = P(A) \times P(B)$$

That equation might look simple, but it says something powerful:

- Knowing A tells you **nothing** about B
- Knowing B tells you **nothing** about A

They are completely separate in behavior. And that's exactly what ICA is searching for.

The Core Idea of ICA - Now let's describe the structure. We assume that what we observe is not the truth. It is a **mixture**. We have:

- Observed data: $X = (x_1, x_2, \dots, x_m)^T$
- Hidden (true) sources: $S = (s_1, s_2, \dots, s_m)^T$

And here's the key assumption: The observed data X is a **linear combination** of the hidden sources S . So ICA asks: "**Can we reverse this process?**"

The Transformation - ICA introduces a matrix: $S = WX$

Where: W is called the **unmixing matrix**. It is learned directly from the data. Its job is to **undo the mixing**.

Think of it like this:

- Reality: Sources $\rightarrow$ get mixed $\rightarrow$ become observations
- ICA: Observations $\rightarrow$ get unmixed $\rightarrow$ recover sources

Why Non-Gaussian Matters - Here's something subtle, but extremely important. ICA doesn't just look for independence directly (that's hard). Instead, it uses a clever trick: ***Independent signals tend to be non-Gaussian.***

So ICA searches for components that are as **non-Gaussian** as possible. Because that strongly suggests independence. This idea becomes the backbone of most ICA algorithms.

Where is ICA actually used?

Because this sounds abstract, until you see it in action. ICA shows up in places where signals are mixed:

- **Audio processing** $\rightarrow$ separating voices (the "cocktail party problem")
- **Image analysis** $\rightarrow$ extracting underlying patterns
- **Biomedical signals** $\rightarrow$ isolating brain activity (EEG), heart signals, etc.

In all these cases, the goal is the same: **Find the hidden sources inside messy, mixed data.**

So if you had to summarize ICA in one sentence:

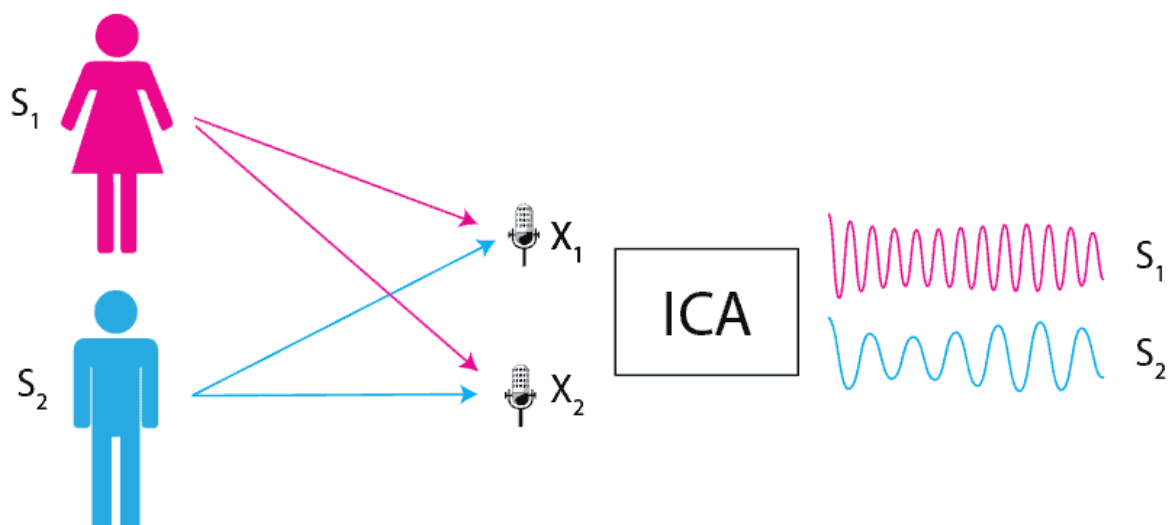
PCA finds directions of maximum variance → ICA finds **independent sources behind the data**

Not just different. Not just uncorrelated. But truly **independent**.

2. Independent Component Analysis – The Cocktail Party Problem

Now let's make ICA feel real. Because everything we've said so far— independence, mixtures, transformations. It's all a bit abstract. So let's go back to a simple (and famous) idea: **The Cocktail Party Problem**.

(The following image is for reference only)



Imagine this. You walk into a room. A group of students are talking. Conversations overlap. Voices mix. And around the room, there are microphones recording everything. Here's the situation:

- There are n **students** speaking → There are n **microphones** recording

- But here's the catch: Each microphone does **not** capture a single voice. Instead, it records a **mixture of all voices**

Breaking it down - We define:

- Independent sources (the real voices): $s_1, s_2, \dots, s_n$
- Observed signals (what microphones hear): $x_1, x_2, \dots, x_n$

So even if one student speaks clearly. Every microphone still hears a blend of everyone. No clean signal. No direct access to the truth. Only mixtures.

The Mixing Process - Mathematically, this mixing is written as: $X = AS$

Where:

- S : original independent voices;
- A : **mixing matrix** (how signals get combined);
- X : what we actually observe

Think of A as the "room effect": Distance between speakers and microphones, echo, sound overlap. All of that gets encoded into A .

The Real Challenge - Here's where things get interesting. In reality, we don't know:

- The original signals S and the mixing matrix A

All we have is: The observed data X → So the problem becomes: **"Can we recover the original voices... using only the mixtures?"**

The ICA Solution - ICA approaches this by learning a matrix W such that: $S = WX$

Here: W is the **unmixing matrix** → Ideally, $W \approx A^{-1}$

So instead of understanding how things were mixed. We directly learn how to **undo the mixing**.

A Deeper Look (Behind the Scenes) - In practice, ICA doesn't jump straight to W . It usually breaks the process into steps:

$$W = RV \Rightarrow S = RVX$$

Or equivalently: $S = RZ$ → Where: $Z = VX_c$ is the **whitened data**, and $V = D^{-1/2}E^T$

→ E : eigenvectors; D : eigenvalues; $D^{-1/2}$: scales data to remove correlations

Then: R is a **rotation matrix** → It satisfies: $RR^T = I$

So the idea becomes: Remove correlation (whitening) → Rotate the data → Maximize independence → All through matrix multiplications.

What Do We Get at the End? - After applying W :

- Each recovered signal corresponds to **one student's voice**
- The signals are **statistically independent**

But there's a small twist: The order may change → The scale may change

And that's completely fine. ICA doesn't care about exact labels. It cares about **recovering independence**.

The Big Idea - So in simple terms: The world gives us **mixed signals**. ICA learns how to **separate them**. Almost like focusing on one voice in a noisy room.

3. Independent Component Analysis – A Simple Dataset (Voice Signals)

Let's make this concrete. We will work with a small dataset of voice signals:

meanfreq	sd	median	label
0.16	0.08	0.19	female
0.18	0.07	0.20	female
0.20	0.06	0.21	female
0.20	0.07	0.20	female
0.21	0.06	0.22	female
0.17	0.07	0.19	female
0.06	0.06	0.03	male
0.07	0.07	0.04	male
0.08	0.08	0.04	male
0.15	0.07	0.16	male
0.14	0.08	0.12	male
0.13	0.08	0.12	male

We represent it as a matrix:

$$X = \begin{bmatrix} 0.16 & 0.08 & 0.19 \\ 0.18 & 0.07 & 0.20 \\ 0.20 & 0.06 & 0.21 \\ 0.20 & 0.07 & 0.20 \\ 0.21 & 0.06 & 0.22 \\ 0.17 & 0.07 & 0.19 \\ 0.06 & 0.06 & 0.03 \\ 0.07 & 0.07 & 0.04 \\ 0.08 & 0.08 & 0.04 \\ 0.15 & 0.07 & 0.16 \\ 0.14 & 0.08 & 0.12 \\ 0.13 & 0.08 & 0.12 \end{bmatrix}$$

Where: Each **row** = one voice sample → Each **column** = one observed signal (feature)

And now, we're ready to begin.

Step 1: Centering the Data (Zero Mean)

Before ICA can even begin doing anything "intelligent". We need to prepare the data. Because ICA is picky. It assumes something very specific:

Each feature must have mean = 0

So the first step is simple: **Center the data.**

What does "centering" mean? We compute the mean of each column:

$$\mu = [0.145, 0.071, 0.143]$$

Then we subtract this mean from every data point: $X_c = X - \mu$

Let's look at one example (first row):

$$[0.16, 0.08, 0.19] - [0.145, 0.071, 0.143] = [0.015, 0.009, 0.047]$$

And we do this for every row.

After Centering - We get the centered dataset:

meanfreq	sd	median
0.0142	0.0092	0.0467
0.0342	-0.0008	0.0567
0.0542	-0.0108	0.0667
0.0542	-0.0008	0.0567
0.0642	-0.0108	0.0767
0.0242	-0.0008	0.0467
-0.0858	-0.0108	-0.1133
-0.0758	-0.0008	-0.1033
-0.0658	0.0092	-0.1033
0.0042	-0.0008	0.0167
-0.0058	0.0092	-0.0233
-0.0158	0.0092	-0.0233

And now something important happens: Each column has **mean** = 0

But we're not done. Centering only removes the mean. It does **NOT** remove relationships between features. So the data is still correlated and uneven in variance. And that's a problem. Because ICA requires something stronger: $\text{Cov}(Z) = I$

Meaning: No correlation → Unit variance → Perfectly "balanced" data. So we need another step.

Step 2: Whitening (Making Data "Clean")

Now we take things further. Whitening is where we transform the data so that: Features become **uncorrelated** and variance becomes **1**

- **Step 2.1: Look at the Covariance** - From the centered data, we compute:

$$\Sigma = \begin{bmatrix} 0.0027 & -0.000096 & 0.00367 \\ -0.000096 & 0.000063 & -0.000121 \\ 0.00367 & -0.000121 & 0.0052 \end{bmatrix}$$

And immediately, we notice: Strong correlation between *meanfreq* and *median* → Very small variance for *sd* → Clearly, not suitable for ICA yet

- **Step 2.2: Eigen Decomposition**

To fix this, we decompose the covariance:

$$\Sigma = EDE^T$$

Where: E are eigenvectors (directions) $\rightarrow D$ are eigenvalues (variances)

From computation:

$$\text{Eigenvalues : } D = \begin{bmatrix} 0.0078 & 0 & 0 \\ 0 & 0.0001 & 0 \\ 0 & 0 & 0.0001 \end{bmatrix}$$

$$\text{Eigenvectors : } E = \begin{bmatrix} -0.584 & 0.6551 & 0.4793 \\ 0.02 & -0.5787 & 0.8153 \\ -0.8115 & -0.4857 & -0.3249 \end{bmatrix}$$

- **Step 2.3: Build the Whitening Matrix** - We scale using inverse square roots:

$$D^{-1/2} = \begin{bmatrix} 11.33 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 100 \end{bmatrix}$$

Then: $V = D^{-1/2}E^T \rightarrow$ After computation:

$$V = \begin{bmatrix} -6.612 & 0.2264 & -9.1884 \\ 65.51 & -57.87 & -48.57 \\ 47.93 & 81.53 & -32.49 \end{bmatrix}$$

- **Step 2.4: Apply Whitening**

Now we transform the data: $Z = X_c V^T \rightarrow$ This is just matrix multiplication (row by row). As a result:

$$Z = \begin{bmatrix} -0.5213 & -2.2348 & -0.1170 \\ -0.7478 & -0.5584 & -0.3625 \\ -0.9744 & 1.1181 & -0.6081 \\ -0.8802 & 1.0070 & 0.9347 \\ -1.1325 & 1.3205 & -0.3991 \\ -0.5897 & -0.7607 & -0.5716 \\ 1.6071 & 0.6064 & -1.7759 \\ 1.4512 & 0.1173 & -0.4637 \\ 1.3873 & 0.2086 & 1.2882 \\ -0.1815 & -0.5851 & -0.5500 \\ 0.2547 & 0.2621 & 1.6631 \\ 0.3209 & -0.5206 & 1.0144 \end{bmatrix}$$

What did whitening actually achieve? Now: Mean is still 0. Variance is 1. Features are **uncorrelated**. In other words: $E[Z] = 0$, $\text{Cov}(Z) = I \rightarrow$ This is exactly what ICA needs.

Step 3: From Whitening $\rightarrow$ Independence

Now comes the final idea. After whitening... everything "simple" is gone: No mean. No correlation. No variance differences. So what's left? Only **higher-order structure**. And that's where ICA operates.

We now find a rotation: $S = RZ$

Where: R is an **orthogonal matrix**. It does not change covariance. But it changes **distribution shape**

The Goal - We want: Components that are **maximally non-Gaussian**. Because that implies **independence**

A common measure:

$$\text{kurt}(s) = E[s^4] - 3(E[s^2])^2$$

Since data is whitened:

$$\text{kurt}(s) = E[s^4] - 3$$

So ICA solves:

$$\max_{\|r\|=1} |E[(r^T Z)^4] - 3|$$

That's the heart of ICA.

- **Step 4: FastICA (How We Actually Solve It)**

Now, how do we find R ? We don't brute-force it. We use an iterative method:

FastICA

Update rule:

$$r_{\text{new}} = E[Z \cdot g(r^T Z)] - E[g'(r^T Z)]r$$

Common choice: $g(u) = u^3$, $g'(u) = 3u^2$

What is happening here? Each iteration: Adjusts direction $r \rightarrow$ Increases non-Gaussianity $\rightarrow$ Moves closer to independence. Until it converges.

Now we can see the full flow:

Centering (remove mean) $\rightarrow$ Whitening (remove correlation) $\rightarrow$ Rotation (ICA) $\rightarrow$ enforce independence

And at the end: We recover the hidden sources just like separating voices in a noisy room. Not perfectly labeled. Not perfectly ordered. But **independent**. And that is exactly what ICA was built to do.

4. Independent Component Analysis with Python

The Scikit-Learn library in Python supports ICA via the **FastICA** method.

We will use the full version of the gender voice dataset:

```
gender_voice.csv
```

(You can run the full program on any Python environment to see the output)

```
import pandas as pd

# Load the dataset
df = pd.read_csv('gender_voice.csv')
```

```
# Fill in missing value with mean
df = df.fillna(df.drop(columns=['label']).mean())
# Mapping
df = df.replace({"male": 0, "female": 1})
```

```
# Find X_centered
columns = df.drop(columns=['label']).columns
X = df[columns]
# Column-wise mean
mu = X.mean(axis=0)
# Centered data (demonstration only)
X_centered = X - mu
print("X_c:\n", X_centered)
```

```
# FastICA automatically performs whitening, then finds the
rotation matrix R.
from sklearn.decomposition import FastICA

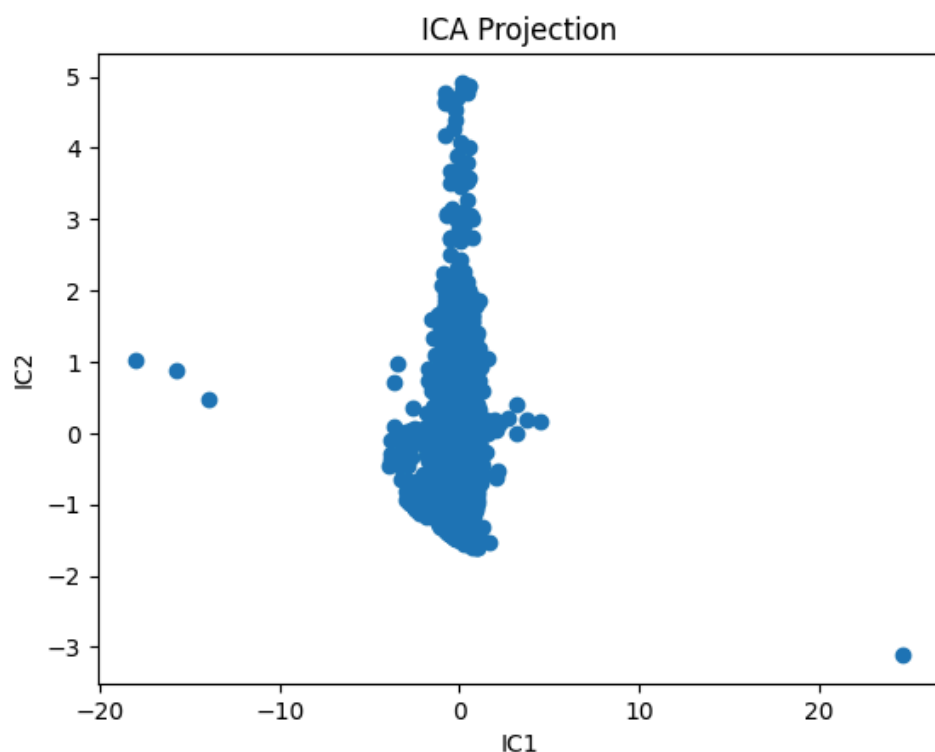
ica = FastICA(n_components=3, whiten='unit-variance', random_state=0)

# Get the FastICA components
# S = RZ -> Independent components
S = ica.fit_transform(X_centered)
print("Independent components (S):\n", S)
# A = Mixing matrix
A = ica.mixing_
print("Mixing matrix A:\n", A)
# Whitening matrix V = D^{-1/2} E^T
V = ica.whitening_
print("Whitening matrix (V):\n", V)
# Whitened data Z = X_c V^T
Z = X_centered.values @ V.T
print("Whitened data (Z):\n", Z)
# Unmixing matrix W = RV
W = ica.components_
print("Unmixing Matrix (W):\n", W)
```

```
# Rotation matrix  $R = W V^T$ 
# Because FastICA uses orthonormal whitening:  $V^{-1} = V^T$ 
R = W @ V.T
print("Rotation matrix (R):\n", R)
```

```
import matplotlib.pyplot as plt

# Store independent components
df_ica = pd.DataFrame(S, columns=["IC1", "IC2", "IC3"])
# Scatter plot of ICA components
plt.scatter(S[:, 0], S[:, 1])
plt.xlabel("IC1")
plt.ylabel("IC2")
plt.title("ICA Projection")
plt.show()
```



Summary

ICA is kind of funny when you think about it. The problem it tries to solve is actually very simple:

“We have mixed signals... can we separate them?”

That’s it. No prediction. No classification. Just **unmixing what’s already there**.

But the moment you look under the hood. Things get messy real fast. Matrices. Eigenvalues. Whitening. Rotations. Optimization. It escalates quickly.

But here’s the important part: **You are not expected to do all of that by hand**.

Because in practice? We use tools. We use libraries. We use implementations like `FastICA`. And they handle all the heavy computation for us.

So what should you actually remember? Not the math. Not the derivations. But the **core idea**:

- The world gives us **mixed data** → ICA assumes hidden sources are **independent** → And it learns how to **separate them**

Almost like focusing on one voice, in a room full of noise.

So the math looks intimidating. But the idea? Surprisingly simple. And like many things in Machine Learning: **Simple idea** → **complicated execution** → **powerful result**.

References

<https://www.geeksforgeeks.org/machine-learning/ml-independent-component-analysis/>

<https://www.youtube.com/watch?v=GgLaP4Des1Q>

Next Section:

 [28. Support Vector Machines](#)